

C++ 연습 문제: 07. 객체 수명, RAII와 소유권

1. C++에서 자원의 획득과 해제를 객체의 수명과 연결하여, 범위를 벗어나거나 예외가 발생해도 소멸자가 자동으로 자원을 정리하도록 하는 기법은 무엇인가? (하)

1. RTTI
2. RAII
3. SFINAE
4. CRTP

2. 동적 할당된 메모리를 관리할 때 `new`와 `delete`를 직접 사용하는 것의 문제점으로 올바른 것은? (하)

1. 컴파일 속도가 현저히 느려진다.
2. 메모리 누수를 방지하기 위해 가비지 컬렉터가 강제로 실행된다.
3. 함수 중간에 `return`하거나 예외가 발생하면 `delete`가 실행되지 않아 메모리가 누수될 수 있다.
4. 포인터의 타입이 암시적으로 변환되어 데이터 손실이 일어난다.

3. 다음 빈칸 (A)에 들어갈 알맞은 스마트 포인터 타입은? (하) [자원에 대한 독점적인 소유권을 가지며, 복사할 수 없고 소유권을 이동(Move)만 할 수 있는 스마트 포인터는 (A)이다.]

1. `std::shared_ptr`
2. `std::unique_ptr`
3. `std::weak_ptr`
4. `std::auto_ptr`

4. `std::unique_ptr`를 다른 함수나 변수로 넘겨주어 소유권을 완전히 이전할 때 사용하는 함수는 무엇인가? (중)

1. `std::copy()`
2. `std::move()`
3. `std::forward()`
4. `std::swap()`

5. 다음 중 `std::unique_ptr` 객체를 생성할 때 가장 권장되는 안전한 방법은 무엇인가? (중)

1. `std::unique_ptr<int> ptr = new int(10);`
2. `std::unique_ptr<int> ptr(new int(10));`
3. `std::unique_ptr<int> ptr = std::make_unique<int>(10);`
4. `std::unique_ptr<int> ptr = std::allocate_unique<int>(10);`

6. 자원에 대한 소유권을 여러 객체가 공유할 수 있으며, 자원을 참조하는 객체의 수가 0이 될 때 메모리를 자동으로 해제하는 스마트 포인터는 무엇인가? (하)

1. `std::shared_ptr`
2. `std::unique_ptr`
3. `std::weak_ptr`
4. `std::scoped_ptr`

7. `std::shared_ptr` 내부에서 자원을 참조하는 객체의 개수(참조 횟수)를 추적하기 위해 생성하는 구조를 무엇이라고 하는가? (상)

1. 가상 함수 테이블(Virtual Table)
2. 제어 블록(Control Block)
3. 해시 테이블(Hash Table)
4. 예외 테이블(Exception Table)

8. 다음 빈칸 (A)에 들어갈 알맞은 함수는? (중) [`std::shared_ptr`를 생성할 때 `new` 연산자를 직접 사용하면 객체와 제어 블록을 위해 메모리를 두 번 할당하게 된다. 이를 한번의 메모리 할당으로 최적화하려면 (A) 함수를 사용해야 한다.]

1. `std::make_shared`
2. `std::make_unique`
3. `std::shared_from_this`
4. `std::allocate_shared`

9. 두 개 이상의 `std::shared_ptr`가 서로를 가리켜 참조 횟수가 영원히 0이 되지 않고 메모리 누수가 발생하는 현상을 무엇이라고 하는가? (하)

1. 버퍼 오버플로(Buffer Overflow)
2. 순환 참조(Circular Reference)
3. 이중 해제(Double Free)
4. 댕글링 포인터(Dangling Pointer)

10. 순환 참조 문제를 해결하기 위해 고안되었으며, 참조 횟수를 증가시키지 않고 `std::shared_ptr`가 관리하는 자원을 관찰만 하는 스마트 포인터는 무엇인가? (하)

1. `std::unique_ptr`
2. `std::weak_ptr`
3. `std::raw_ptr`
4. `std::observer_ptr`

11. `std::weak_ptr`가 가리키는 자원이 아직 유효한지 확인하고, 유효하다면 안전하게 접근하기 위해 `std::shared_ptr` 객체를 반환해 주는 멤버 함수는 무엇인가? (상)

1. `get()`
2. `acquire()`
3. `lock()`
4. `share()`

12. 부모 노드와 자식 노드의 관계로 트리 구조를 설계할 때, 순환 참조로 인한 메모리 누수를 방지하기 위한 소유권 정책으로 가장 올바른 것은? (상)

1. 부모와 자식 모두 서로를 `std::unique_ptr`로 소유한다.
2. 부모와 자식 모두 서로를 `std::shared_ptr`로 소유한다.
3. 부모는 자식을 `std::weak_ptr`로 관찰하고, 자식은 부모를 `std::shared_ptr`로 소유한다.
4. 부모는 자식을 `std::shared_ptr`로 소유하고, 자식은 부모를 `std::weak_ptr`로 관찰한다.

13. 클래스 내부에서 자기 자신을 가리키는 `std::shared_ptr`를 안전하게 생성하여 반환하기 위해 상속받아야 하는 표준 라이브러리 클래스는 무엇인가? (상)

1. `std::enable_shared_from_this`
2. `std::make_shared_from_this`
3. `std::shared_ptr_provider`
4. `std::enable_weak_from_this`

14. 프로그램 실행 중 예외(Exception)가 발생했을 때, `catch` 블록으로 제어권이 이동하면서 호출 스택에 생성되어 있던 지역 객체들의 소멸자를 자동으로 호출하여 자원을 정리하는 과정을 무엇이라고 하는가? (상)

1. 스택 오버플로(Stack Overflow)
2. 스택 풀링(Stack Pooling)
3. 스택 되감기(Stack Unwinding)
4. 스택 트레이스(Stack Trace)

15. C++에서 예외 처리에 대한 설명으로 옳지 않은 것은? (중)

1. 예외를 발생시킬 때는 `throw` 키워드를 사용한다.
2. 예외를 처리할 때는 `try` 블록 안에서 코드를 실행하고 `catch` 블록에서 예외를 잡는다.
3. 소멸자 내부에서 예외를 밖으로 던지(`throw`)는 것은 자원 누수를 막기 위해 강력히 권장된다.
4. `catch` 블록을 여러 개 작성할 때는 파생 클래스(구체적인 예외)부터 기반 클래스(일반화된 예외) 순서로 작성해야 한다.

16. 여러 종류의 예외를 처리하는 다음 코드에서 빈칸 (A)에 들어갈 가장 적절한 표준 타입은 무엇인가? (중)

```
C++
try {
    // 자원 로드 작업 수행
}
catch (const std::invalid_argument& e) {
    // 잘못된 인수 예외 처리
}
```

```
catch (const ( A )& e) {
    // 그 외의 모든 표준 예외를 마지막으로 포착하여 처리
}
```

1. `std::exception`
2. `std::error`
3. `std::runtime_error`
4. `std::bad_alloc`

17. 함수의 매개변수로 유효하지 않은 값(예: 빈 리소스 이름)이 전달되었을 때, 이를 외부에 알리기 위해 발생시키기에 가장 적절한 표준 예외는 무엇인가? (하)

1. `std::out_of_range`
2. `std::invalid_argument`
3. `std::bad_cast`
4. `std::length_error`

18. 배열이나 컨테이너의 범위를 벗어난 인덱스에 접근하려고 시도할 때 주로 발생하는 표준 예외는 무엇인가? (하)

1. `std::out_of_range`
2. `std::domain_error`
3. `std::logic_error`
4. `std::bad_typeid`

19. 다음 빈칸 (A)에 들어갈 알맞은 낱말은? (중) [다형성을 올바르게 유지하고 불필요한 객체 복사를 피하기 위해, 예외 객체를 `catch` 블록으로 전달받을 때는 값(Value)이 아닌 (A)(으)로 받아야 한다.]

1. 포인터(Pointer)
2. 참조(Reference)
3. 우측값(rvalue)
4. 전방 선언(Forward Declaration)

20. 스마트 포인터와 RAII를 활용한 예외 및 자원 관리에 대한 설명으로 올바른 것은? (중)

1. `std::unique_ptr`를 사용하면 별도의 가비지 컬렉터 프로그램이 실행되어야 하므로 속도가 크게 저하된다.
2. RAII 기법은 파일이나 네트워크 소켓 같은 운영체제 자원에는 적용할 수 없고, 오직 동적 메모리에만 적용 가능하다.
3. 생성자에서 두 개의 자원을 순서대로 획득하다가 두 번째 자원에서 예외가 발생하면, 첫 번째 자원은 RAII 객체로 관리했을 때 안전하게 자동 해제된다.
4. 예외가 발생하여 실행 흐름이 강제로 변경되면, 스마트 포인터에 보관된 동적 메모리는 해제되지 않고 반드시 메모리 누수가 발생한다.